Introduction and Overview

This strategy can be summarized into two parts: a pricing function and a system that continuously trades from the current price to the target price. We combine a general macro view using a computationally light data-set (OHLCV) for the overall direction of trades, and then a computationally heavy data-set (orderbook) for execution.

The idea is simple: use macro-concepts to determine directional bias and parse noise, use speed instead of straight leverage to achieve returns beyond the risk-free rate.

The algorithm is done in steps:

1. The Macro Signal: $W_{macro}(t) \in [-1, +1]$ at bar boundaries and smooth it.
2. Micro Signal: $A_{micro}(t) \in [-1, +1]$ every evaluation tick from the order book.
3. Bias Merge: $bias(t) = clip(\alpha W_{macro}(t) + (1 - \alpha)A_{micro}(t), -1, +1)$
4. Bias > Target: $targetposition(t) = round(bias(t) \cdot maxposition)$
5. Trade towards the target with a single contract deadzone.
    a. Deadzone is used to reduce churn.

The data that is required includes:

1. Macro Inputs

This is just OHLCV bars in whichever candle timeframe we choose. As this is primarily used for a macro-bias, we do not need anything more granular than close data.

2. Micro Inputs

At the very minimum, we require the best bid and ask, as well as the queue size, for the top-of-market price. Additionally, the algorithm benefits from deeper orderbook data in order to compute deeper orderbook imbalances (OBI)

Signal Construction

The macro-signal is constructed through a weighted mix of three subscores:

1. EMA crossover momentum
2. Bollinger mean reversion vs breakout
3. Some form of a trend filter

EMA Crossover Momentum: w_{ma} normalized momentum.

We use fast and slow EMAs computed on closes, then take the difference normalized by recent vol, and use the result to avoid large leverage slips or huge spreads. The idea is that if we normalize by the volatility, the signal strength becomes comparable across different market regimes because markets like to give us outlier prices and spikes.

Bollinger Subscore: w_{boll} regime-decider.

After the Bollinger bands are computed, the strategy decides between the regimes using a short-term ratio of realized volatility vs the band sigma. If the vol is relatively low, the strategy uses a mean-reverting behavior near our bounds. Otherwise, it behaves like a breakout. The idea is that Bollinger signals don't always need to be mean-reverting, and the dual-regime system reduces the usual failures that come from vol-trading.

Trend Filter: w_{trend} checks and balances.

The idea for the EMAs is that the slope of the slow EMA is estimated and normalized by volatility, then constrained within a narrow range. It doesn't dominate the overall trend; instead, it penalizes macro signals that run counter to our determined trend.

Combining the Macro Signal: $W_{macro}(t) = sW_{raw} + (1 - s)W_{macro}(t - 1)$

The raw macro value is a weighted sum of the subscores that we outlined. Then there is a damping term for counter-trend signals (we multiply it by 0.7 when the raw sign conflicts with the trend), then there is clamping, and then the equation becomes the one given above. The idea is that the macro-trend is slow, and with smooth a lot of otherwise flip-flopping signals and prevents the micro from getting overridden by a spiking macro term.

The micro signal combines the weighted OBI across the depth we have, then uses the microprice deviations at the immediate touch, then we scale it by a confidence term based on the spread at the top of the book.

Weighted OBI Term: $OBI = \frac{BidVol - AskVol}{BidVol + AskVol}$ and clamped to [-1,+1]
The idea is that as prices and liquidity move further from the touch-prices, their weightage should theoretically matter less, but it still matters, hence the weightage.

The microprice deviation is computed using the NBBO compared to the mid deviation, which is normalized by the spread. The idea being that the microprice compartmentalizes our short-term direction estimator and normalizes the spread, making it comparable.

The confidence term reduces the micro-influence if the OB becomes untrustworthy for any reason, such as when the volume is thin or the spread is enormous. We implemented this through a confidence scalar $c \in [0,\ 1]$ applied to the blended OBI signal. Microstructure signals are best when the OB is tight and liquid; hence, our algorithm stops if it behaves aggressively when the market is not providing sufficient-quality data.

The Micro Alpha Equation: $A_{micro} = clip((0.6 \cdot OBI + 0.4 \cdot microdeviation) \cdot c, -1, +1)$

The final step for signal generation is to blend the macro and micro through linearization with a parameter α, or the macro_weight_alpha.

The bias is converted to a target price using max_position. The strategy rebalances by trading the delta between the current price and the target price, subject to a single contract deadzone. If the linear blending and the sizing are simple, they would become easily testable and easy to port. There are many more complex strategies out there that utilize extremely complicated mathematics, but this strategy allows us to isolate other variables and look at their effects.

Risk Management

While the algorithm is in a position, the position is flattened if any of the following triggers occur:

1. Stop loss
2. Take profit
3. Max holding time

The exit check runs before the retargeting on the same tick of evaluation.

Additionally, we included a daily_loss_limit term to act as a killswitch if the algorithm bombs on any day.

We also modeled a rudimentary version of slippage for whether our position fills or not, beyond the basics of queue positioning:

- Buy fills at ask+slippage_ticks*tick_size
- Sell fills at bid-slippage_ticks*tick_size

This is to bake in slippage before we even begin to look at latency.

We looked at some design limitations to show that we also know where this algorithm clearly lacks, but that is an improvement for another day. Here is what we have initially seen:

- Our linearization is probably suboptimal if we run into 2+ sigma regimes, but that is done intentionally for stable testing (and what the killswitch is for)
- Micro inputs seem to be very asset-dependent, as well as dependent on the day it is trading. This algorithm has built-in kill switches to ensure that when the strategy faces uncertainty that is outside of its parameters, it just stops. A better strategy would be able to adapt to those outlier events.
- Retargeting will sometimes cause overtrade if our parameters are tuned too aggressively. However, as an HFT algorithm, it fundamentally conflicts if we place cooldowns on the execution level.

Portability

We also provided a pseudocode that describes the exact update order:

1. Update macro at completed bars
2. Compute micro at every tick
3. Blend the bias and target
4. Check exits
5. Retarget

```
state:
  W_macro = 0
  latest_bar_time = None

on_each_eval_tick(t, book):
  # --- 1) Update macro at bar closes ---
  bar = last_completed_bar_at_or_before(t)
  if bar exists and bar.time > latest_bar_time:
    W_raw = macro_signal_from_history(bar_history_up_to(t))
    W_macro = s*W_raw + (1-s)*W_macro
    latest_bar_time = bar.time

  # --- 2) Micro alpha each tick ---
  OBI = weighted_imbalance(book.depth_levels)
  micro_dev = (microprice(book.bbo) - mid(book.bbo)) / spread(book.bbo)
  c = confidence_from_spread_and_depth(book.bbo)
  A_micro = clip((0.6*OBI + 0.4*micro_dev)*c, -1, +1)

  # --- 3) Combine & target position ---
  bias = clip(alpha*W_macro + (1-alpha)*A_micro, -1, +1)
  target_pos = round(bias * max_position)

  # --- 4) Check exits first ---
  if position != 0:
    if stop_loss_hit() or take_profit_hit() or hold_time_exceeded():
      flatten_position()

  # --- 5) Re-target ---
```

```
delta = target_pos - position
if abs(delta) >= 1:
  send_order_to_change_position_by(delta)  # obey position limits
```